

# 可验证产出总结

FAUST CTF 2026 (国际 Attack/Defense, 502 支队伍, 8 小时) 参赛队伍 **qiyu** · FAUST 队号 **980** · 最终排名 **67 / 502**

本页所有数字都可点开核验。核验方法见文末「可核验性是怎么设计的」一节。全部产出为公开仓库：  
<<https://github.com/nnn493368-max/faust-ctf-2026-writeups>>

## 一、结果

| 指标            | 数值                                                            | 核验                     |
|---------------|---------------------------------------------------------------|------------------------|
| 总排名           | <b>67 / 502 (前 13.3%)</b>                                     | <a href="#">官方冻结数据</a> |
| 总分            | <b>10,752.81</b>                                              | 同上                     |
| 进攻 / 防守 / SLA | 6,737.16 / -5,069.72 / 9,085.37                               | 同上                     |
| CTFtime 记录    | Place 67 · Rating 15.167                                      | <a href="#">队伍页</a>    |
| 四个服务终态        | 全部 <code>status = 0 (up)</code> ，无 <code>down / faulty</code> | 同上                     |

**进攻分构成：**四个服务里只有一个真正被打穿，但它贡献了几乎全部分数——

| 服务                              | 进攻分             | 占比           |
|---------------------------------|-----------------|--------------|
| <b>IMC</b> (NekoVM + SQLite)    | <b>6,726.31</b> | <b>99.8%</b> |
| Alf (Flask + 自定义 C 扩展)          | 10.85           | 0.2%         |
| Lamp (XeLaTeX 写的"Web 框架")       | 0               | —            |
| Rufflecopter (Rust + Flash 模拟器) | 0               | —            |

表中「0 分」的两个服务我**没有包装**。它们的完整逆向/漏洞分析都在下面，失败的边界也写清楚了。

## 二、交付物

### 1. 公开技术报告 (3 篇, 624 行)

| 报告                          | 主题                                | 链接                              |
|-----------------------------|-----------------------------------|---------------------------------|
| <b>IMC 三漏洞链</b>             | 源码审计 → 跨租户读旗原语                    | <a href="#">imc.md</a>          |
| <b>Rufflecopter AVM2 逆向</b> | 纯字节码逆出一个用 ActionScript 写的 HTTP 服务 | <a href="#">rufflecopter.md</a> |
| <b>Lamp TeX 头注入</b>         | 无认证任意文件读取 + 4 条被证伪的枚举路径           | <a href="#">lamp.md</a>         |

均为**中文全文** + **English abstract**。

2. 可运行代码

| 文件                          | 行数  | 说明                                              |
|-----------------------------|-----|-------------------------------------------------|
| exploits/imc.py             | 490 | 得分引擎；含差分预言机与官方 flag id 快速路径                     |
| exploits/_receipt_decode.py | 51  | <b>自包含</b> 的三平面 QR 解码器（内嵌 1079 个下标 + 4698 字节基底） |
| exploits/lamp.py            | 150 | 任意文件读取                                          |
| exploits/alf.py             | 132 | 堆泄露利用                                           |
| exploits/rufflecopter.py    | 128 | 协议模块（逆向完整，无可用原语）                                |
| exploits/_template.py       | 75  | 模块接口                                            |

统一接口，可直接替换目标与 flag id 复现：

```
NAME = "imc"; PORTS = [8080]
def run(target, flag_ids, timeout) -> list[str] # 返回 FAUST_... 旗标
```

3. 参与证明链

| 环节             | 存档                                 |
|----------------|------------------------------------|
| 官方冻结数据         | <a href="#">scoreboard.json 快照</a> |
| 队伍 → 成绩（含成员名单） | <a href="#">team/449998 快照</a>     |
| 个人 → 队伍        | <a href="#">user/275322 快照</a>     |

三、三个可深聊的技术点

① IMC：三个缺陷串成一条链（占 99.8% 进攻分）

三个 bug 全都**不是“过滤不严”**，而是控制流与语义层面的问题 —— 这是靠读源码、不是靠模糊测试找出来的：

| # | 缺陷                                                | 为什么致命                                                                                                           |
|---|---------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| 1 | <b>SQLite 双引号标识符回退</b> ：与列名同名的双引号串被解析成 <b>列引用</b> | SET crew = "age" 实际是 crew = age。数据库是 STRICT 表、crew 是 INTEGER —— 按常理字符串会被类型系统拦下，但经这条回退， <b>赋的是合法整数</b> ，类型约束反而放行 |
| 2 | <b>catch 块缺 return</b>                            | 邀请校验抛异常后被吞掉，控制流 <b>穿透</b> 到它本该拦下的那条 UPDATE                                                                      |
| 3 | <b>过滤谓词缺 != null 守卫</b> （同一谓词在另一个文件里有这个守卫）        | Neko 里 null == null 为真 → 一次 FETCH 拿走 <b>全部</b> 旗船                                                               |

**两个漏洞的组合尤其值得说**：更新逻辑按列顺序遍历，所以一条 descriptor 能同时写 age 和 crew —— age 是攻击者完全可控的整数，于是 crew = age = 任意值，一条请求就伪装成任意船队成员。

**额外做的**：一个基于 LIKE 的差分预言机。输入白名单**禁掉了 % 却放过了 \_**，所以做不了任意长度通配 —— 但可以做**定长模式**匹配，足够逐位枚举 id，避免爆破。

② Rufflecopter: 纯 AVM2 逆向 (技术上最硬)

这个服务的 HTTP 服务端逻辑**整个写在 SWF 里**, Rust 只负责跑模拟器 —— 常规 Web 套路 (翻 JS、找路由表、看中间件) 全部失效, 只能读 AVM2 字节码。

踩到并记录的两个方法论坑:

- 第一版反汇编器只 dump 方法体, 星球坐标怎么也找不到 —— **类级静态数据不在方法体里**
- 坐标也不在 ABC 常量池, 而在 **SWF 主时间轴的 PlaceObject3 变换矩阵里**

→ 「程序状态可以存在于时间轴/显示列表里」, 只读 ABC 块永远找不到

逆出的机制:

- 路由 = **2D 空间邻近碰撞**, 不是查表。碰撞半径 `r1 = Spaceship.width/4*0` **恒等于 0** (死代码), 实际由 `r2 = planet.width/2+1 = 8.5px` 决定
- 鉴权是**其中一个"星球"**, 不是中间件 → 攻击面变成**导航问题**
- **三平面 QR 隐写解码器**: `(receiptcode[i] ^ base[i]) & 7` 一次取出三个位平面。

因为写入是顺序的、**没有 QR 标准的 zigzag 交织**, 所以**不需要实现 Reed-Solomon**, 纯位运算即可

然后诚实的部分: 我用几何算出一条 **42px 宽的缝隙**, 写了弹道模拟器穷举, 找到**两条完全不经过门禁星球的航线** —— **但读不到旗子**。因为路径同时驱动"渲染哪个页面"和"用哪个角度飞", `/receipts` 被硬编码在 60°, 从该起点必然飞出网格。

结论从"必经门禁"被我自己推翻, 最后变成**"路径与角度被同一张表绑定"**。我把这个**自我修正过程保留在报告里** —— 一个正确的结论配一个错误的理由, 等于没有理解。

③ Lamp: 定位到"校验作用于错误的值"

`http.sty` 用**请求头的名字定义宏**, 于是 `Path:` 头覆盖了框架内部的 `\httpPath`; 而目录穿越检查跑在**解析请求头之前**, 只作用于请求行。

提炼成一句可复用的原则:

**校验必须作用于最终生效的值。先校验再覆盖, 等于没校验。**

原语在真实靶机上验证成功 (读到 `/srv/main.tex`、`/etc/passwd`), 但未能转化为分数, 原因如实写明: 文件名是每队随机的 16 位 hex, **4 条枚举路径全部证伪** —— 其中一条 (NUL 截断) 能让服务崩溃, **我主动放弃**, 因为它既违规又会被 SLA 扣分。

四、工程能力

| 项     | 数据                                                                                 |
|-------|------------------------------------------------------------------------------------|
| 攻击编排  | 并发 8 → <b>24</b> ; 单轮耗时 <b>380s → 130s</b> (原先一轮超过一个 tick, 永远追不上节奏)                |
| 产出效率  | 唯一旗速率 <b>6.0/分钟 → 18.8/分钟 (3.1×)</b> ; 单轮提交质量 OK 141~166 / 过期 5~12                 |
| 可靠性   | 看门狗每 45s 探测四个服务, 游戏地址不可达即自动重启容器; 一次真实的 <code>docker-proxy</code> 转置故障就是这么恢复的       |
| 发布流水线 | <code>preflight.py</code> <b>四项自动自检</b> (脱敏 / 数字对齐 / 链接可达 / 编号一致性) + 一键发布脚本 + 存档脚本 |

**发布流水线这一段是我最想强调的**: 它把"文档里的数字"变成了**可被机器校验的东西**。 `preflight.py` 会拿 README 里的每个数字去比对官方 `scoreboard-full.json`, 不一致就非零退出 —— 所以**文档不可能和代码/数据漂移**。

## 五、工程判断与边界

这部分我认为比上面任何一段都更能说明问题。

### 明确拒绝做的三件事

1. **不做 DoS** —— 发现 NUL 截断能打崩服务后立即停手
2. **不越授权边界** —— 只打比赛分配给自己的网段，不碰基础设施网段
3. **不写未验证的结论** —— 所有“已验证”的说法都附了复现方式

### 我推翻过自己的结论，并且保留了过程

- Rufflecopter 的最终结论理由被我自己用几何分析推翻并重写（见上）
- 记分板文档里一处百分比把“击败比例”写成了“前百分之几”，被指出后修正
- 一次“快照里没有成员名单”的判断是**从规则推理出来的、没有验证**，实抓后发现自己错了，

这条教训写进了文档：「快照有没有抓到某段内容，要去把快照抓下来看」

**两个 0 分的服务如实写明**，包括“为什么没打下来”和“卡在哪一步”。对安全岗位来说，**能准确说出攻击面在哪里失效，比含糊的“打不动”有价值得多。**

## 六、可核验性是怎么设计的

这一层是我刻意加的，也是这份产出和“自己写个博客说我很强”的区别：

| 手段            | 作用                                                                                                                                        |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| 官方数据 + 第三方时间戳 | 主办方发布、Internet Archive 存档 —— <b>不依赖任何当事方的自述</b>                                                                                           |
| SHA256 互锁     | 仓库内 <code>data/scoreboard-full.json</code> 与存档副本 <b>字节完全相同</b> ：<br><br> 673b0b475c6a07928f8645ce437a3c74c9bb129b8d7d23588c0a9d7f8433c98f |
| preflight.py  | 数字、链接、脱敏、编号一致性全部机器校验                                                                                                                      |
| commit 固定存档   | 分支快照会随 push 过期，所以同时存档 commit URL（内容不可变）                                                                                                   |
| 区分三个编号        | 队号 980（网络编址）/ 记分板 id 185（内部主键）/ CTFtime 449998 —— 三者不同命名空间，文档里显式对照，避免误读                                                                   |

## Summary (English)

**FAUST CTF 2026** — international Attack/Defense, 502 teams, 8 hours. Team **qiyu** finished **67 / 502** (top 13.3%, 10,752.81 pts).

I reversed and exploited all four services. One of them, **IMC** (NekoVM + SQLite), produced **99.8% of our offense**: three chained defects found by reading the provided sources — SQLite's double-quoted-token fallback (a quoted column name resolves as a `*column reference*`, so `SET crew = "age"` really assigns `crew = age` and slips past the `STRICT` typing), a `catch` block missing its `return` (control falls through the invitation check), and a fetch filter missing a `!= null` guard (Neko evaluates `null == null` as true, so one request leaks every flag ship). I added a `LIKE`-based differential oracle: the input whitelist blocks `%` but allows `_`, giving fixed-length pattern matching and digit-by-digit id enumeration.

The hardest target, **Rufflecopter**, turned out to implement its whole HTTP server inside a Flash SWF, so it had to be reversed from AVM2 bytecode — including a self-written disassembler, a three-plane QR steganogram decoder (no Reed-Solomon

needed, as there is no zigzag interleaving), and a ballistic simulator that found two routes bypassing the only auth gate. It still could not be weaponised, and the writeup states exactly why.

All three writeups are public, every exploit ships, and the results are backed by third-party-timestamped snapshots of the organisers' own data plus a machine-checkable SHA256 — so every number here can be independently verified.

---

\*所有链接均为公开可访问。参与证明的完整方法见仓库内 `EVIDENCE.md` 。 \*